CSCI 264 — Assembly Language

Lab 1

Base Conversion of Integers in C

This assignment exercises number base conversion, the ASCII code, and null-terminated strings in C. Last week the C library converted an integer to octal for you with a single `%o` in a format string. This week you do the work yourself.

Warm-up (by hand, on paper)

Show the division steps, not just the answers.

1. Convert 422 to base 2.
2. Convert 9742 to base 16.

If you can do these by hand, you already know the algorithm your program has to implement. If you cannot, ask about it in lab before you start coding.

Programming Assignment

Write a C program which reads a non-negative decimal integer and a base, and prints the representation of that number in that base. Your program must work for any base from 2 through 36.

Some hints

Recall the repeated division approach discussed in lecture. Given `u` is the number being converted and `b` is the base:

```
r = u % b;
u = u / b;
```

A loop which continues until `u` reaches zero, will yield each remainder `r` as one digit of the answer.

A remainder is a *number* between 0 and 35. To print it you need the corresponding *character*, and in C a character is just a small integer holding an ASCII code. The two rules you need:

- For digit values 0 through 9, the character code is the value plus 48.
- For digit values 10 through 35, use the letters A through Z: the character code is the value plus 55.

You may also refer to the ASCII table

Output

Store the characters in an array of `char`:

```
char y[100];
```

This has one problem: The algorithm produces the digits in the wrong order. The first remainder you compute is the *last* digit of the result and the last remainder you compute is the first digit. If you store the characters in the array in the order you calculate them, starting at index 0, the number prints out backwards.

For this assignment, fix it the obvious way: once the loop is finished, copy the characters into a second array in reverse order and print that.
After the last character you must store a zero. That zero is the *null terminator*, and it is how the C library knows where your string stops. Print the finished string with:

```
printf("%s\n", y);
```

Test cases

Your program should reproduce all of these:

Input	Base	Output
100	8	144
422	2	110100110
9742	16	260E
1000	7	2626
35	36	Z
0	2	0

Submission

Submit your source file on Canvas by Tuesday, September 8 and demo the program to Professor West by the end of the next lab session. Put your name in a comment at the top of the source file. If a classmate helped you put their name in the comment.